

DAA Assignment 2

Abhiram Aravind [Sr No 25238]

1 JE Chapter 3 Question 1(c)

Input: Dream-Dollar denominations $C = \{\$1, \$4, \$7, \$13, \$28, \$52, \$91, \$365\}$, value $\$k$.

Output: Minimum number of Dream-Dollar notes needed to make $\$k$.

Constraint: Time complexity polynomial in k (pseudo-polynomial)

Take $18 = 14 + 4 = 7 + 7 + 4$. We observe that if $7 + 7 + 4$ is the optimal way to make 18, then $7 + 7$ must be the optimal way to make 14. Generalising, for any value k , if $a + b = a_1 + a_2 + \dots + a_m + b_1 + b_2 + \dots + b_n$ is the optimal way to make k , then $a_1 + a_2 + \dots + a_m$ must be the optimal way to make a . Otherwise, if $a = a'_1 + a'_2 + \dots + a'_{m'}$, with $m' < m$ is a better way to make a , then $k = a'_1 + a'_2 + \dots + a'_{m'} + b_1 + b_2 + \dots + b_n$ would be a better way to make k , violating optimality.

So, if $X(k)$ is the minimum number of notes to make $\$k$, then:

$$X(k) = \begin{cases} 1 & \text{if } k \in C \\ \min_{0 < a < k} X(a) + X(k - a) & \text{if } k \notin C \end{cases}$$

With this in mind, we can design an algorithm to compute $X(k)$ as follows:

1. Define array $A[1 \dots k]$ for memoisation.
2. Initialise $A[1 \dots k]$ with ∞ .
3. For i in $1 \dots k$:
 - (a) If $i \in C$, set $A[i] = 1$.
 - (b) Else for j in $1 \dots \lfloor \frac{i}{2} \rfloor$:
 - i. Set $A[i] = \min(A[i], A[j] + A[i - j])$.
4. Return $A[k]$.

Analysis:

- Run-time complexity (assuming constant C) $O(k^2)$
- Space-time complexity $O(k)$

2 JE Chapter 3 Question 3(b)

Input: Array $A[1 \dots n]$ of real numbers.

Output: The maximum product of sub-arrays.

We define the sub-problem $\text{Prod}(i)$ as returning a triple $\langle p, g, m \rangle$ where:

- $p = \max_{i \leq t \leq n} \prod_{k=i}^t A[k]$:- The maximum product possible for a subarray beginning at i .
- $g = \min_{i \leq t \leq n} \prod_{k=i}^t A[k]$:- The minimum product possible for a subarray beginning at i .
- $m = \max_{i \leq a, b \leq n} \prod_{k=a}^b -A[k]$:- The maximum product for a subarray not necessarily beginning at i .

The idea is that if $x < y$ and $a < 0$, then $ax > ay$. Hence, we need to keep track of both the minimum and maximum product sub-arrays.

We define $\text{Prod}(i) = \langle 1, 1, 1 \rangle$ when $i > n$. When $i \leq n$, we define $\text{Prod}(i)$ recursively. Suppose $\text{Prod}(i+1) = \langle p', g', m' \rangle$, then $\text{Prod}(i) = \langle p, g, m \rangle$ where:

- $p = \max(A[i], A[i] * p', A[i] * g')$
- $g = \min(A[i], A[i] * g', A[i] * p')$
- $m = \max(p, m')$

Finally, the m value returned by $\text{Prod}(1)$ gives the maximum product of the entire array A . Implementing it as an algorithm would be as follows:

1. Set $p = g = m = 1$.
2. For i from n to 1:
 - (a) Set $q = p$, as we need the present value of p after updating it.
 - (b) Set $p = \max(A[i], A[i] * p, A[i] * g)$.
 - (c) Set $g = \min(A[i], A[i] * g, A[i] * q)$.
 - (d) Set $m = \max(p, m)$.
3. Return m .

As is obvious, the algorithm has a runtime of $O(n)$.

3 JE Chapter 8 Question 14

Input: A weighted directed graph G , two source vertices s_0, s_1 . Weights $w(u \rightarrow v)$ are non-negative.

Output: Vertex x that minimises the maximum of the weights of the paths from s_0 and s_1 .

We modify the Dijkstra's algorithm to do the following:

1. Initialise $F_0 = F_1 = \phi$.
2. Create an empty priority queue Q .
3. Set $d_0(s_0) = d_1(s_1) = 0$
4. $Q.\text{Add}(\text{item: } \langle s_0, 0 \rangle, \text{priority: } 0); Q.\text{Add}(\text{item: } \langle s_1, 1 \rangle, \text{priority: } 0);$
5. While $Q \neq \phi$
 - (a) $\langle u, i \rangle = Q.\text{ExtractMin}()$; Note $i \in \{0, 1\}$
 - (b) If $u \in F_{1-i}$, return $x = u$.
 - (c) Set $F_i := F_i \cup \{u\}$
 - (d) For all edges $u \rightarrow v$ where $v \notin F_i$:
 - i. Set $d_i(v) = \min(d_i(v), d_i(u) + w(u \rightarrow v))$
 - ii. $Q.\text{UpdateOrAdd}(\text{item: } \langle v, i \rangle, \text{priority: } d_i(v))$

Essentially, the algorithm above performs two instances of Dijkstra's algorithm using s_0 and s_1 as sources, sharing the same priority queue, and terminating when they reach the same point x . Let us call them the two 'branches' of the algorithm. We note that the algorithm shares the same run time complexity as Dijkstra's algorithm $O(|E| + |V| \log |V|)$ using a Fibonacci heap.

If $\delta_0(x)$ is the weight of the shortest path from s_0 to x and $\delta_1(x)$ the weight of the shortest path from s_1 to x , then we see the following:

Claim 3.1. x minimises $\max(\delta_0(x), \delta_1(x))$

Proof. Assume our algorithm got it wrong and vertex y is the correct vertex with $\max(\delta_0(x), \delta_1(x)) > \max(\delta_0(y), \delta_1(y))$. We analyse the following three cases:

- y is neither on path $s_0 \cdots x$ nor on path $s_1 \cdots x$: By the optimality of Dijkstra's algorithm, $\delta_0(x) \leq \delta_0(y)$ and $\delta_1(x) \leq \delta_1(y) \implies \max(\delta_0(x), \delta_1(x)) \leq \max(\delta_0(y), \delta_1(y))$, a contradiction.
- y is on path $s_0 \cdots x$: The s_0 branch of Dijkstra's algorithm would reach y before x . By the optimality of Dijkstra's algorithm, we see $\delta_0(y) < \delta_0(x)$ and $\delta_1(y) > \delta_1(x)$. Now suppose $\delta_1(y) < \delta_0(x)$. Since both branches of Dijkstra's algorithm share the same priority queue, this would imply that the s_1 branch would reach y before x , thus making y the first vertex both branches reach first, which violates the assumption that x was the first vertex reached by both branches $\implies \delta_0(x) \leq \delta_1(y) \implies \max(\delta_0(x), \delta_1(x)) \leq \max(\delta_0(y), \delta_1(y))$, a contradiction.
- y is on path $s_1 \cdots x$: A similar argument as above shows $\max(\delta_0(x), \delta_1(x)) \leq \max(\delta_0(y), \delta_1(y))$, a contradiction.

□

4 JE Chapter 8 Question 16

Input: A weighted directed graph G with non-negative weights $w(u \rightarrow v)$, two vertices s and t .

Output: The shortest walk from s to t with number of edges divisible by 3.

Definition 4.1. We define a weighted directed graph G^* as the following:

- For every vertex v in G , G^* contains three vertices v^0, v^1, v^2 .
- For every edge $u \rightarrow v$ in G , G^* contains three edges of the same weight: $u^0 \rightarrow v^1, u^1 \rightarrow v^2, u^2 \rightarrow v^0$.
- V^i is the set of all vertices in G^* with superscript i .

Claim 4.2. If $s^0, v_1^1, v_2^2, v_3^0 \dots t^0$ is the shortest path from s^0 to t^0 in G^* , then $s, v_1, v_2, v_3, \dots t$ is the shortest walk from s to t in G whose number of edges is divisible by 3.

Proof. One observes that all vertices in V^i have edges leading to vertices in $V^{(i+1) \bmod 3}$. Therefore, any path between two vertices in V^i must have number of edges divisible by 3. Now if $s, v_1, v_2, v_3, \dots t$ is the shortest walk from s to t in G whose number of edges is divisible by 3, then by definition of G^* , there exists edges $s^0 \rightarrow v_1^1, v_1^1 \rightarrow v_2^2, v_2^2 \rightarrow v_3^0$ and so on. Since the number of edges in the original path is divisible by 3, the new path in G^* must terminate in a vertex in V^0 , and hence in t^0 . Therefore, the shortest path from s^0 to t^0 in G^* would correspond to the shortest walk from s to t in G with number of edges divisible by 3. $\square$

Algorithmically, one can simulate finding the shortest path in G^* using Dijkstra's algorithm by running the Dijkstra's algorithm on G but appending the vertices with their superscripts. That is, $\langle v, i \rangle \equiv v^i$

1. Set $S = \phi$.
2. Create empty priority queue Q .
3. $Q.\text{Add}(\text{item: } \langle s, 0 \rangle, \text{priority: } 0)$;
4. Set $d(\langle s, 0 \rangle) = 0$
5. While $Q \neq \phi$
 - (a) $\langle u, i \rangle = Q.\text{ExtractMin}()$; Note $i \in \{0, 1, 2\}$
 - (b) If $u = t$ and $i = 0$, terminate.
 - (c) Set $S = S \cup \{\langle u, i \rangle\}$
 - (d) Let $j = (i + 1) \bmod 3$
 - (e) For all edges $u \rightarrow v$ where $\langle v, j \rangle \notin S$
 - i. Set $d(\langle v, j \rangle) = \min(d(\langle v, j \rangle), d(\langle u, i \rangle) + w(u \rightarrow v))$
 - ii. $Q.\text{UpdateOrAdd}(\text{item: } \langle v, j \rangle, \text{priority: } d(\langle v, j \rangle))$

Since G^* has only a constant factor more edges and vertices than G , the algorithm maintains the runtime of $O(|E| + |V| \log |V|)$ if one uses a Fibonacci heap.

5 JE Chapter 9 Question 6(b)

Input: Weighted directed graph G with weight matrix $\text{Exch}[1..n, 1..n]$. Weights are positive.

Output: Whether an arbitrage cycle exists, i.e. a cycle $v_1 v_2 v_3 \cdots v_k v_1$ where $\text{Exch}[v_1, v_2] * \text{Exch}[v_2, v_3] * \cdots * \text{Exch}[v_k, v_1] < 1$

Let $\ln(G)$ be the resulting graph when one takes the logarithm of the weights in G . Suppose $\text{LnEx}[1..n, 1..n]$ be the weight matrix of G' , with

$$\text{LnEx}[i, j] = \ln(\text{Exch}[i, j])$$

We see that an arbitrage cycle exists in G if and only if a negative cycle exists in $\ln(G)$.

$$\text{Exch}[v_1, v_2] * \text{Exch}[v_2, v_3] * \cdots * \text{Exch}[v_k, v_1] < 1$$

$$\ln(\text{Exch}[v_1, v_2] * \text{Exch}[v_2, v_3] * \cdots * \text{Exch}[v_k, v_1]) < \ln(1)$$

$$\ln(\text{Exch}[v_1, v_2]) + \ln(\text{Exch}[v_2, v_3]) + \cdots + \ln(\text{Exch}[v_k, v_1]) < 0$$

$$\text{LnEx}[v_1, v_2] + \text{LnEx}[v_2, v_3] + \cdots + \text{LnEx}[v_k, v_1] < 0$$

The Floyd-Warshall algorithm can be used to find negative cycles as follows:

1. Initialise $d(i, j) = \text{LnEx}[i, j] \ \forall 1 \leq i, j \leq n$
2. For $1 \leq i \leq n$
 - (a) For $1 \leq j \leq n$
 - i. For $1 \leq k \leq n$
 - A. Set $d(j, k) = \min(d(j, k), d(j, i) + d(i, k))$
3. If $d(i, i) < \text{LnEx}[i, i]$ for some i , then there exists a negative cycle in $\ln(G)$.

$\ln(G)$ can be calculated in $O(n^2)$ time and the Floyd-Warshall algorithm takes $O(n^3)$ time, resulting in a total runtime of $O(n^3)$. In fact, one can perform the Floyd-Warshall algorithm directly on the original graph G if one uses $d(j, k) = \min(d(j, k), d(j, i) * d(i, k))$ instead to update $d(j, k)$.